CTF CHEAT SHEET

YOUR TOOLS — WHAT EACH DOES & HOW IT WORKS

🚩 **FLAG BOX** Where you type the answer you have found.

Type the flag exactly, keeping the `flag{ }` wrapper, then press Submit. Capitals don't matter — spelling and underscores do.

</> **VIEW HTML** Shows the raw code behind a web challenge.

Click it on a challenge and read anything between `<!--` and `-->` (comments) or inside `<script>` tags. It replaces Inspect Element, which school PCs block.

🧰 **DECODER** Undoes encodings and ciphers — your main crypto tool.

Press **Send to Decoder**, then add operations (From Base64, From Hex, ROT13, XOR...). Stack them in order; the output updates as you go. Re-read it after each step and choose the next.

🔢 **HEX VIEWER** Shows the real bytes of a file and the text inside it.

Open a file in it. The first bytes (the **magic number**) reveal the true file type even if the name lies, and the **Strings** list often contains `flag{`.

🖼️ **IMAGE INSPECTOR** Reveals data hidden inside a picture.

Open an image and try **Exaggerate tiny differences** and the single colour channels (**Red / Green / Blue only**). Writing hidden in near-identical colours jumps out. Check the metadata too.

💻 **JS SCRATCHPAD** Runs small bits of code for programming & logic puzzles.

Type a little JavaScript (such as a loop), press **Run**, and read the output. Perfect for brute-forcing a number or adding a long list up.

#️⃣ **HASHES TOOL** Identifies and cracks simple password hashes.

Paste a hash (a fixed-length string of 0-9 and a-f). It names the likely type and tries common passwords from a wordlist.

KEY WORDS YOU'LL HEAR

Flag
The secret you are hunting, written like `flag{…}`.

CTF
Capture The Flag — a cyber-security puzzle competition.

Encoding
A reversible way of writing data (Base64, hex). No key needed.

Encryption
Scrambling data with a key — you need the key to read it.

Cipher
A method of encryption, such as Caesar or Vigenère.

Hash
A one-way fingerprint of data. You can't reverse it, only guess and check.

Brute force
Trying every possibility until one works (e.g. all 25 Caesar shifts).

Metadata
Hidden data carried inside a file, like a photo's camera or comment.

Magic number
The first bytes of a file that reveal its true type.

Steganography
Hiding a message inside something else, such as an image.

OSINT
Open Source Intelligence — facts found from public information.

Payload
The useful data part — for example the middle of a login token.

CTF CHEAT SHEET

SPOT IT & SOLVE IT

SPOT THE ENCODING

What the data looks like, what it probably is, and what to reach for.

IF YOU SEE…	IT'S PROBABLY…	USE
Letters/numbers ending in `=`	**Base64**	`Decoder → From Base64`
Only 0-9 and a-f, in pairs	**Hex**	`Decoder → From Hex`
Only 0 and 1, groups of 8	**Binary**	`Decoder → From Binary`
Small numbers like 102 108	**ASCII codes**	`Decoder → From Decimal`
Words almost readable, all shifted	**Caesar / ROT**	`Decoder → Caesar brute force`
Letters mirrored (z, y, x common)	**Atbash**	`Decoder → Atbash`
The challenge names a keyword	**Vigenère**	`Decoder → Vigenère decode + key`
Random bytes + the word "key"	**XOR**	`Decoder → From Hex, then XOR`
Right letters, wrong order	**Rail fence**	`Decoder → Rail fence decode`
Dots and dashes	**Morse**	`Decoder → Morse`
Numbers 1-26	**A1Z26**	`Decoder → A1Z26`
A dark or blank-looking picture	**Steganography**	`Decoder → Image inspector`

DECODER OPERATIONS

From Base64 / Hex / Binary / Decimal — Turn encoded data back into text.

ROT13 — Shift every letter 13 places (do it twice to undo).

Caesar brute force — Show all 25 shifts; pick the readable one.

Atbash — Mirror the alphabet (A↔Z).

XOR with key / brute force — Undo a XOR using a key, or try all 255 single bytes.

Vigenère decode — Undo a keyword cipher (type the key).

Rail fence decode — Un-zig-zag a transposition (set the rails).

Reverse — Flip the text back to front.

Morse / A1Z26 — Dots-and-dashes, or numbers 1-26, back to letters.

Golden rules. 1) Every flag looks like `flag{…}` — keep the wrapper. 2) Work out *what* the data is before decoding. 3) For layered puzzles, decode one step, look again, then choose the next. 4) Stuck for 15 minutes? Move on and come back.